xPegy

AI E-Bike Fleet Manager

A hands-on AI workshop

90 minutes · Hands-on

How today works

The practical contract

- **Short theory block** (~2 min per concept) I show you a **recorded demo** — real prompts, real outputs **YOU do it** :

```
/module-1
...
/module-4
```

- No compiling, no SDKs — **one app is all you need**

Roadmap

Time	Phase	What happens
10'	Setup	Install & first chat
15'	How AI works	LLM, models, tokens, context, effort
15'	Module 1	Context window
15'	Module 2	Grounding & hooks
15'	Module 3	Custom tools
15'	Module 4	Autonomous agent
5'	Wrap-up	The big picture

What to install… and what NOT

You need exactly TWO things:

1. **OpenCode Desktop — Windows (x64)** → `opencode.ai/download`
2. **Url** → `http://bit.ly/4bcO6xk`
3. **This repo** → `git clone https://github.com/JaviPege/mundus-ai-programming-course` or GitHub → Code → Download ZIP

Then: open `starter/` in OpenCode · pick a free model (**Big Pickle**)

NOT needed: NO compiler · NO SDK · NO Python · NO API key · NO account · NO billing

We will not compile any code — the agent IS the only tool you need.

Check it works

1. Open `starter/` → new session → send `Hello` → it answers ✅
2. Stuck later? Type **`/module-1`** … **`/module-4`** — the AI tutors you step by step
3. The `demos/` folders = the instructor's recorded demos (not for you… yet)

What an LLM actually does

Text in → probability machine → text out.

- It does NOT "know" things — it predicts plausible text
- It does NOT remember you — no memory of its own
- It cannot DO anything — every action is code running for it

Models

Not all models are equal:

- Big ↔ small · fast ↔ smart · free ↔ paid
- Today: FREE models inside OpenCode —
- **Big Pickle** & friends
- Pick yours in the **model selector** (bottom of the window)

Tokens

Models don't read words — they read **tokens**

- 1 token ≈ 4 characters ≈ ¾ of a word
- Tokens = the "currency" of every model call
- Every call bills input + output — long conversations cost more

```
"Battery not charging"  →  ~5 tokens
```

Context window & statelessness

The context window = **the model's entire world**

- The ONLY thing the model sees is the text in the current session
- Every call starts from ZERO — no memory between calls
- "Memory" = the app resending the conversation every time

Reasoning effort

Models can "think longer" before answering

- **More effort** → better reasoning… slower, more tokens
- **Less effort** → fast and cheap… more mistakes

- Crank it up for hard problems; keep it fast for chat

Module 1 · Context in practice

- A **session** IS the context window — its "memory”

- New session = **total amnesia**

  `/compact` — summarizes long history so it fits

  `AGENTS.md` = the project's persistent memory (read every session)

Module 1 · DEMO: an agent with NOTHING

`demos/m1-bare` → session **DEMO M1 — No tools: the model invents the fleet report**

```
Give me the morning status report for the e-bike fleet: total bikes,
broken bikes, and battery levels. Just give me the report, no questions.
```

- Confident, detailed… and **100% invented** (200 bikes?! the real fleet has 8) **"how would you know?"**

Then YOU: **`/module-1`**

Module 2 · Grounding & hooks

- **Grounding:** give the model the FACTS —

  `AGENTS.md` rule + read tools = the simplest RAG

- **Hooks:** middleware that intercepts tool calls BEFORE they run

  Our guard **blocks dangerous commands — deterministically**

Module 2 · DEMO: same prompt, different output

`demos/m2-grounded` → session **DEMO M2 — Same prompt, grounded: real data via fleet tools**

The SAME prompt as M1 → `fleet_get_fleet_status` → **real data: 8 bikes · #3, #7 broken · #5 maintenance**

Same prompt, different output — **grounding, not a smarter model**

LIVE:

`run rm -rf /tmp/whatever` → the guard **blocks it** ✅

Then YOU: **`/module-2`**

Module 3 · Function calling

- A **tool** = YOUR function, described to the model (name + Zod schema = the contract)
- The model **PROPOSES** the call:

```
fleet_mark_bike_fixed {"bike_id": 3}
```

- YOUR code **executes** it — a real `UPDATE` via `sqlite`
- **The model proposes; your code disposes.**

Module 3 · DEMO: a sentence writes to the DB

`demos/m3-tools` → session **DEMO M3 — Function calling: 'Bike #3 is repaired' writes to the DB**

```
Bike #3 is repaired.
```

- One plain sentence → **`fleet_mark_bike_fixed {"bike_id": 3}`**
- The DB actually changed: bike #3 → `ok`, its tickets → `closed`

Then YOU: `/module-3`

Module 4 · Agents

- **Chatbot:** you talk, it answers.
- **Agent:** you give a GOAL, it works
- The loop: **observe → decide → act → repeat**

  `steps: 20` = the safety fuse · permissions:

- **ask / allow / deny**

Module 4 · DEMO: let it loose

`demos/m4-agent` → session **DEMO M4 — Autonomous agent: resolves all tickets in a loop**

```
Resolve all pending maintenance tickets
```

- The loop: **list → assign → fix ×3** (ticket #4 closes along the way)
- It **STOPS BY ITSELF** when none remain ✅

Then YOU: **`/module-4`**

Today you built

Memory (context) · **Truth** (grounding) · **Hands** (tools) · **Autonomy** (agents)

That is 90% of modern AI engineering.

Now go build.

Questions?

Workshop materials: github repo · opencode.ai

xPegy